```
unbound = MyModule.instance_method(:my_method)
unbound.class              # => UnboundMethod
```

You can't call an UnboundMethod, but you can use it to generate a normal method that you can call. You do that by binding the UnboundMethod to an object with UnboundMethod#bind. UnboundMethods that come from a class can only be bound to objects of the same class (or a subclass), while UnboundMethods that come from a module have no such limitation from Ruby 2.0 onward. You can also bind an UnboundMethod by passing it to Module#define_method, as in the next example:

```
String.class_eval do
  define_method :another_method, unbound
end

"abc".another_method  # => 42
```

UnboundMethods are used only in very special cases. Let's look at one of those.

The Active Support Example

The Active Support gem contains, among other utilities, a set of classes and modules that automatically load a Ruby file when you use a constant defined in that file. This "autoloading" system includes a module named Loadable that redefines the standard Kernel#load method. If a class includes Loadable, then Loadable#load gets lower than Kernel#load on its chain of ancestors—so a call to load will end up in Loadable#load.

In some cases, you might want to remove autoloading from a class that has already included Loadable. In other words, you want to stop using Loadable#load and go back to the plain vanilla Kernel#load. Ruby has no uninclude method, so you cannot remove Loadable from your ancestors once you have included it. Active Support works around this problem with a single line of code:

gems/activesupport-4.1.0/lib/active_support/dependencies.rb

```
module Loadable
  def self.exclude_from(base)
    base.class_eval { define_method(:load, Kernel.instance_method(:load)) }
  end

  # ...
```

Imagine that you have a MyClass class that includes Loadable. When you call Loadable.exclude_from(MyClass), the code above calls instance_method to get the original Kernel#load as an UnboundMethod. Then it uses that UnboundMethod to define a brand-new load method directly on MyClass. As a result, MyClass#load is actually the